SUMMER RESEARCH INTERNSHIP REPORT

Computer Log Anomaly Detection Using Machine Learning Techniques

A Comparative Study Using Machine Learning and Deep Learning Models

Submitted by

Rajol Kumar

Roll No.: 25MAC2R19

Under the Guidance of

Dr. Sri Vallabha Deevi

Adjunct Faculty

Wadhwani School of Data Science & Artificial Intelligence (WSAI)

Indian Institute of Technology Madras

Summer Research Internship

2026

Certificate

This is to certify that the internship report entitled **Computer Log Anomaly Detection Using Machine Learning Techniques**, submitted by **Rajol Kumar** (Roll No. 25MAC2R19), is a genuine record of the research work carried out during the Summer Research Internship at the Wadhwani School of Data Science and Artificial Intelligence (WSAI), Indian Institute of Technology Madras, under the guidance of Dr. Sri Vallabha Deevi.

The work presented in this report includes the design, implementation, evaluation, and comparative analysis of machine learning and deep learning methods for network traffic anomaly detection. To the best of my knowledge, this work has not been submitted elsewhere for the award of any degree, diploma, or certificate.

Dr. Sri Vallabha Deevi
Adjunct Faculty
Wadhwani School of Data Science & AI
Indian Institute of Technology Madras

Acknowledgement

I express my sincere gratitude to my mentor, Dr. Sri Vallabha Deevi, for providing me with the opportunity to undertake this Summer Research Internship at the Wadhwani School of Data Science and Artificial Intelligence, Indian Institute of Technology Madras. His technical guidance, valuable suggestions, and constructive feedback were instrumental throughout the project. The discussions held during the internship helped me understand not only how machine learning models are implemented, but also how an experimental study should be designed so that its conclusions remain scientifically meaningful.

The internship gave me the opportunity to work with realistic network-traffic datasets containing hundreds of thousands to more than one million observations. Through this work, I gained practical experience in data preprocessing, exploratory analysis, supervised and unsupervised machine learning, Deep Autoencoders, feature selection, hyperparameter optimisation, threshold calibration, and evaluation under severe class imbalance. I am also grateful to the faculty members and internship coordinators of WSAI for providing an excellent research environment and the computational resources required to complete this study.

Finally, I would like to thank my family and friends for their constant encouragement, patience, and support throughout the internship period. Their support helped me remain focused during the experimental and documentation stages of the project.

Rajol Kumar

Abstract

Modern computer networks generate enormous volumes of communication data, while cyber attacks continue to evolve in both scale and complexity. Signature-based intrusion detection systems remain useful for recognising previously known attacks, but their dependence on predefined signatures limits their ability to identify new or modified attack behaviour. Machine-learning-based anomaly detection provides a complementary approach by learning statistical and behavioural characteristics from network traffic and identifying observations that deviate from the expected distribution [1, 2].

This internship investigates supervised, unsupervised, and deep learning methods for network anomaly detection across datasets of increasing size and complexity. The study begins with a small labelled network-traffic dataset containing 1,000 observations, where supervised classifiers are compared with unsupervised anomaly-detection methods under a leakage-aware train-test protocol. The work then progresses to the UNSW-NB15 benchmark, where seven supervised classifiers are evaluated while preserving the official training and testing partitions. Mutual Information feature selection and Optuna-based hyperparameter optimisation are used during model development, and the selected LightGBM pipeline achieves an F1-score of 0.8921 with Recall of 0.9787 on the official test partition.

A second medium-scale experiment uses the CSE-CIC-IDS2018 `02-15-2018.csv` file to evaluate Multivariate Gaussian modelling, Isolation Forest, One-Class SVM, and a Deep Autoencoder in an unsupervised setting. Separate benign training, calibration, development, and final holdout pools are constructed so that model selection and threshold calibration remain independent of the final evaluation data. The selected Autoencoder achieves an F1-score of 0.8329 with Recall of 1.0000 on the final 5% anomaly holdout.

The final stage increases the benign Autoencoder training set to 300,000 observations. The optimized benign-only model achieves an F1-score of 0.8986, Precision of 0.8171, Recall of 0.9980, and a False Positive Rate of 0.0118 on the final 5% anomaly holdout. A controlled training-contamination study then introduces GoldenEye traffic at 0.2%, 1%, and 5% of fixed-size 300,000-row training sets. Mean final F1 decreases from 0.5952 under clean training to 0.2684, 0.0598, and 0.0067 respectively. These results demonstrate

that Deep Autoencoders can provide effective one-class network anomaly detection when trained on representative benign data, while also showing that reconstruction models are highly sensitive to contamination of the nominally normal training distribution.

Contents

Certificate 1

Acknowledgement 2

Abstract 3

1 Introduction 1

1.1 Introduction . . . 1

1.2 Computer Logs and Network Traffic Logs . . . 1

1.3 Network Anomaly Detection . . . 2

1.4 Motivation . . . 3

1.5 Problem Statement . . . 3

1.6 Objectives . . . 4

1.7 Major Contributions . . . 4

1.8 Report Organization . . . 5

2 Literature Review 6

2.1 Intrusion Detection Systems . . . 6

2.2 Supervised Learning for Network Security . . . 6

2.3 Unsupervised and One-Class Anomaly Detection . . . 7

2.4 Deep Autoencoders . . . 7

2.5 Benchmark Intrusion-Detection Datasets . . . 8

2.6 Research Gap . . . 8

3 Datasets and Experimental Methodology 10

3.1 Datasets Used . . . 10
3.2 Small Dataset . . . 11
3.3 UNSW-NB15 . . . 11
3.4 CSE-CIC-IDS2018 . . . 12
3.5 Handling Missing and Infinite Values . . . 13
3.6 Feature Scaling . . . 13
3.7 Feature Selection . . . 13
3.8 Hyperparameter Optimization . . . 14
3.9 Threshold Calibration . . . 14
3.10 Evaluation Metrics . . . 14

4 Small Dataset Experiments **16**

4.1 Experimental Setup . . . 16
4.2 Supervised Results . . . 16
4.3 Unsupervised Results . . . 17
4.4 Discussion . . . 17

5 Medium Dataset Experiments **18**

5.1 Part A – Supervised Classification on UNSW-NB15 . . . 18
5.1.1 Experimental Design . . . 18
5.1.2 Hyperparameter Optimization . . . 18
5.1.3 Final Official Test Performance . . . 19
5.2 Part B – Unsupervised Anomaly Detection on CSE-CIC-IDS2018 . . . 20
5.2.1 Experimental Setup . . . 20
5.2.2 Models Evaluated . . . 20
5.2.3 Autoencoder Model Selection . . . 20
5.2.4 Final Holdout Results . . . 21
5.3 Comparison of the Medium-Scale Studies . . . 21

6 Large Dataset Experiments **23**

6.1 Introduction . . . 23

6.2 Large-Scale Data Preparation . . . 23

6.3 Benign-Only Training Experiment . . . 24

6.3.1 Training and Calibration . . . 24

6.3.2 Hyperparameter Optimization . . . 24

6.3.3 Final Benign-Only Results . . . 24

6.4 Training Contamination Experiment . . . 25

6.4.1 Motivation . . . 25

6.4.2 Training Dataset Composition . . . 26

6.4.3 Final Contaminated-Training Results . . . 26

6.5 Comparison Between Clean and Contaminated Training . . . 27

6.6 Discussion . . . 27

7 Conclusion and Future Work **29**

7.1 Conclusion . . . 29

7.2 Limitations . . . 30

7.3 Future Work . . . 31

Chapter 1

Introduction

1.1 Introduction

Computer networks have become fundamental to modern society and are used extensively in banking, healthcare, education, transportation, cloud computing, government services, industrial control systems, and personal communication. Increasing dependence on networked infrastructure has simultaneously increased the potential impact of cyber attacks. Malicious network activity may disrupt services, obtain unauthorised access, steal information, consume computational resources, or compromise the confidentiality and integrity of critical systems. Intrusion Detection Systems therefore serve as an important security layer alongside firewalls, access-control mechanisms, encryption, and authentication systems [2].

Traditional intrusion detection frequently depends on attack signatures. In a signature-based system, incoming activity is compared with previously identified malicious patterns. Such systems can provide high reliability against attacks that have already been observed, but their dependence on known signatures limits their ability to detect new, modified, or previously unseen attacks. Anomaly-based intrusion detection provides a complementary strategy by modelling expected behaviour and identifying observations that differ sufficiently from that expected behaviour [1]. Machine learning is especially suitable for this problem because network traffic contains many interacting numerical and categorical features whose relationships are difficult to describe using manually written rules.

1.2 Computer Logs and Network Traffic Logs

Computer logs are chronological records generated by operating systems, servers, applications, databases, network devices, and security infrastructure. They record events such as authentication attempts, system failures, application activity, network communication,

and security-relevant actions. Log analysis is widely used for troubleshooting, operational monitoring, security auditing, and incident investigation [3].

The present internship focuses primarily on network-flow logs. Instead of storing every transmitted packet independently, a network flow summarises communication between endpoints using statistical features. Typical attributes include protocol, source and destination ports, flow duration, packet counts, byte counts, packet-length statistics, inter-arrival times, TCP flags, and transfer rates. These representations provide enough behavioural information for statistical and machine-learning analysis while substantially reducing the dimensionality compared with raw packet streams.

Modern datasets such as UNSW-NB15 and CSE-CIC-IDS2018 provide flow-level representations of benign and malicious network traffic. They are therefore useful for systematically comparing supervised classification, one-class learning, statistical anomaly detection, and deep representation learning [4, 5].

1.3 Network Anomaly Detection

Network anomaly detection attempts to identify observations that differ substantially from expected network behaviour. Depending on the environment, such observations may indicate denial-of-service attacks, malware communication, unauthorised access, data exfiltration, scanning, or other security incidents. The key attraction of anomaly detection is that it does not necessarily require a predefined signature for every possible malicious event.

Machine-learning approaches to anomaly detection may be supervised, semi-supervised, or unsupervised. Supervised algorithms learn directly from labelled normal and attack observations and often provide strong performance when representative labels are available. One-class and unsupervised approaches instead attempt to characterise normal behaviour or identify observations that are statistically isolated from the dominant data distribution [1]. This latter setting is particularly relevant in cybersecurity because new attacks may not have representative labels during model development.

1.4 Motivation

Network anomaly detection is challenging because real traffic exhibits substantial variability even during legitimate operation. The data are also highly imbalanced because attacks typically represent only a small fraction of the observed traffic. Under such conditions, accuracy can be misleading: a model that predicts nearly every observation as benign may achieve high accuracy while detecting almost no attacks. Meaningful evaluation therefore requires Precision, Recall, F1-score, False Positive Rate, and False Negative Rate in addition to overall accuracy.

Another important issue is experimental methodology. Data leakage can occur when scaling parameters, feature-selection scores, resampling operations, thresholds, or hyperparameters are determined using information from the final test set. Such leakage can create unrealistically optimistic results. The experiments in this report are therefore designed around strict separation between training, calibration, model-development, and final evaluation data.

1.5 Problem Statement

Although many machine-learning and deep-learning algorithms have been applied to network intrusion detection, their measured performance depends strongly on dataset composition, preprocessing decisions, feature selection, class imbalance, threshold selection, and training strategy. Supervised models can exploit labelled attacks but may be less suitable when representative labels are unavailable, while one-class models avoid this requirement but depend heavily on the quality of the data used to define normal behaviour.

The central problem addressed in this internship is therefore not simply to identify the algorithm with the highest score, but to examine how different detection paradigms behave under progressively more realistic experimental conditions. Particular emphasis is placed on determining whether a Deep Autoencoder can model benign network traffic effectively at scale and how strongly its anomaly-detection capability changes when attack traffic contaminates the nominally normal training distribution.

1.6 Objectives

The main objectives of the internship are:

1. To study machine-learning and deep-learning techniques for network anomaly detection.
2. To perform systematic preprocessing and exploratory analysis on network-traffic datasets of increasing scale.
3. To compare supervised classification and unsupervised anomaly detection on a small labelled dataset.
4. To evaluate seven supervised classifiers on UNSW-NB15 while preserving the official train-test separation.
5. To compare Multivariate Gaussian modelling, Isolation Forest, One-Class SVM, and Deep Autoencoder methods on CSE-CIC-IDS2018.
6. To investigate the impact of feature selection and hyperparameter optimisation on model performance.
7. To develop a large-scale benign-only Autoencoder using an independent calibration set and untouched final holdouts.
8. To study the robustness of reconstruction-based anomaly detection when the training data contain 0.2%, 1%, and 5% attack contamination.

1.7 Major Contributions

The first contribution of this work is a multi-stage experimental framework in which model complexity and dataset scale are increased gradually. The small dataset is used to establish baseline behaviour; UNSW-NB15 provides a realistic supervised benchmark; CSE-CIC-IDS2018 provides a large one-class anomaly-detection problem; and the final Autoencoder experiments investigate both clean and contaminated training distributions.

The second contribution is the use of an evaluation design intended to reduce information leakage. In the supervised experiments, scaling, feature selection, and resampling are

fitted only using training data. The official UNSW-NB15 test set is preserved throughout development. For the CSE-CIC experiments, model training, threshold calibration, hyperparameter development, and final evaluation use separate data pools. This design produces more conservative but substantially more defensible estimates of model performance.

The third contribution is the training-contamination study. Three fixed-size Autoencoder training conditions are created with 0.2%, 1%, and 5% GoldenEye contamination and compared with clean benign training. The experiment demonstrates that the quality of the reference training distribution is a critical factor in reconstruction-based network anomaly detection.

The complete source code, experiment notebooks, result files, and project documentation are maintained in the public project repository [6].

1.8 Report Organization

This report consists of seven chapters. Chapter 1 introduces the research problem, motivation, objectives, and contributions. Chapter 2 reviews intrusion-detection and anomaly-detection methods. Chapter 3 describes the datasets, preprocessing methodology, data separation, feature selection, hyperparameter optimisation, and evaluation metrics. Chapter 4 presents the small-dataset experiments. Chapter 5 presents supervised classification on UNSW-NB15 and unsupervised anomaly detection on CSE-CIC-IDS2018. Chapter 6 presents the large-scale benign-only Autoencoder and training-contamination experiments. Chapter 7 summarises the main conclusions, limitations, and future research directions.

Chapter 2

Literature Review

2.1 Intrusion Detection Systems

An Intrusion Detection System monitors system or network activity to identify behaviour that may represent malicious use. IDS are commonly divided into signature-based and anomaly-based approaches. Signature-based detection compares observed events with known malicious patterns. Its principal strength is reliable identification of attacks for which high-quality signatures already exist, while its main limitation is reduced coverage of previously unseen or substantially modified attacks [2].

Anomaly-based detection takes a different approach by defining or learning expected behaviour and identifying observations that deviate from it [1]. This increases the possibility of detecting novel attacks but creates a different challenge: legitimate yet unusual traffic can also differ from the learned normal profile and therefore generate false alarms. The practical success of anomaly-based detection consequently depends on both the quality of the model and the representativeness of the reference data.

2.2 Supervised Learning for Network Security

Supervised machine learning assumes that observations are accompanied by labels indicating their target class. In network intrusion detection, a model can therefore learn the relationship between traffic features and normal or malicious labels. Algorithms ranging from Logistic Regression and Support Vector Machines to decision-tree ensembles have been widely used for this purpose.

Tree-based ensemble models are especially effective for structured network-flow data because they can represent nonlinear relationships and interactions without requiring every predictor to follow a linear decision structure. Random Forest uses many decorrelated

decision trees, while XGBoost and LightGBM build boosted tree ensembles that sequentially reduce prediction errors [7, 8, 9]. Their strong predictive capacity makes them useful benchmarks whenever reliable labels are available.

The fundamental limitation is that supervised models depend directly on representative attack labels. A classifier may perform strongly for known attacks represented during training while generalising less effectively to new attack types. This motivates the inclusion of one-class and unsupervised techniques in the present internship.

2.3 Unsupervised and One-Class Anomaly Detection

Unsupervised and one-class methods attempt to identify anomalous observations without requiring labelled attacks during model fitting. Multivariate Gaussian modelling represents normal traffic using estimated distributional parameters and measures how far a new observation lies from the expected multivariate structure. This approach is computationally efficient but relies on assumptions that may be restrictive for complex network data.

Isolation Forest identifies anomalies through recursive random partitioning. Rare or unusual observations generally require fewer random partitions before becoming isolated, allowing path length to serve as an anomaly score [10]. One-Class SVM instead learns a boundary around normal observations and classifies points lying outside that boundary as unusual [11]. These techniques offer different trade-offs between flexibility, scalability, and computational cost.

2.4 Deep Autoencoders

An Autoencoder is a neural network trained to reconstruct its own input through an internal latent representation [12]. It consists of an encoder that maps the original feature vector to a compressed representation and a decoder that reconstructs the input from that representation.

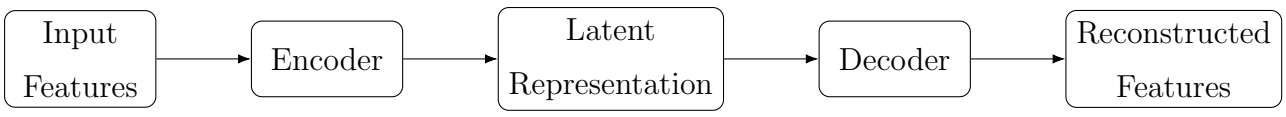


Figure 2.1: Conceptual structure of a reconstruction-based Autoencoder.

When an Autoencoder is trained using representative normal traffic, it is expected to learn the dominant structure of that traffic and reconstruct similar observations with relatively low error. Inputs that differ substantially from the learned representation may produce larger reconstruction errors. Comparing this error with a calibrated threshold therefore provides an anomaly-detection mechanism.

The behaviour of an Autoencoder depends strongly on the training distribution. If malicious observations appear repeatedly during training, the optimization process also encourages the network to reconstruct those observations accurately. Training contamination can therefore reduce the separation between benign and malicious reconstruction errors. This effect motivates the primary large-scale experiment in this report.

2.5 Benchmark Intrusion-Detection Datasets

UNSW-NB15 was developed as a modern intrusion-detection benchmark containing benign network traffic and multiple attack categories [4]. It provides official training and testing partitions, making it suitable for supervised benchmark evaluation. The dataset is used in this internship to study binary Normal-versus-Attack classification.

CSE-CIC-IDS2018 contains large quantities of benign and malicious network traffic represented through flow-level features [5]. The `02-15-2018.csv` capture used in this work contains Benign, DoS-GoldenEye, and DoS-Slowloris observations. Its scale and high-dimensional numerical feature space make it suitable for investigating unsupervised anomaly detection and Deep Autoencoders.

2.6 Research Gap

Many anomaly-detection studies focus primarily on final model scores, while experimental design issues such as preprocessing leakage, threshold calibration, anomaly prevalence, and training contamination may receive less attention. These factors are particularly important for reconstruction-based models because the model, preprocessing pipeline, and threshold can all unintentionally incorporate information from data later reported as test data.

The present study therefore places strong emphasis on data separation. The experiments preserve benchmark test sets where available, learn preprocessing statistics from

training data, use separate calibration sets for threshold selection, reserve independent development data for hyperparameter optimisation, and evaluate the selected models on untouched final holdouts.

Chapter 3

Datasets and Experimental Methodology

3.1 Datasets Used

Three publicly available network-traffic sources are used across the study. The first is a small labelled anomaly-detection dataset containing 1,000 observations. The second is UNSW-NB15, whose official training and testing partitions contain 175,341 and 82,332 records respectively. The third is the CSE-CIC-IDS2018 `02-15-2018.csv` file containing 1,048,575 raw flow records.

The CSE-CIC source file is used twice at different experimental scales. The medium-scale comparison trains several unsupervised models using 100,000 benign observations. The large-scale Autoencoder experiment increases the benign training sample to 300,000 observations and examines the effect of deliberately contaminating that training distribution with GoldenEye traffic.

Table 3.1: Datasets used throughout the internship

Dataset	Scale	Records	Purpose
Network Traffic Anomaly Detection Dataset	Small	1,000	Initial comparison of supervised and unsupervised anomaly detection.
UNSW-NB15	Medium	175,341 train; 82,332 test	Supervised binary classification with feature selection and hyperparameter optimisation.
CSE-CIC-IDS2018 `02-15-2018.csv`	Medium / Large	1,048,575 raw	Unsupervised model comparison, large-scale Autoencoder training, and training-contamination analysis.

3.2 Small Dataset

The small dataset contains 1,000 observations with approximately 90% normal traffic and 10% anomalies. No missing values or duplicate rows were detected. The feature `mean_packet_size` was constant and was therefore removed. Boolean predictors were retained and represented numerically rather than discarded.

A stratified 80:20 train-test split was performed before scaling, feature selection, or oversampling. This ordering prevents information from the test partition from influencing learned preprocessing steps. SMOTE, class weighting, feature selection, and scaling were restricted to the training workflow.

3.3 UNSW-NB15

The official UNSW-NB15 training set contains 175,341 observations, while the official test set contains 82,332 observations. The original dataset contains 45 columns. The `id` column is excluded because it is an identifier, while `attack_cat` is removed from the binary predictor set because it describes the attack category. The binary `label` is retained as the target.

Three categorical predictor fields—`proto`, `service`, and `state`—are one-hot encoded. The official test partition is never combined with the training partition during prepro-

cessing or model selection. Instead, a stratified validation subset is created from the official training partition, and all learned transformations are fitted using that training-side data. The final selected pipeline is subsequently refitted using the complete official training partition before being evaluated once on the official test partition.

3.4 CSE-CIC-IDS2018

The raw `02-15-2018.csv` file contains 1,048,575 observations and 80 columns. Its original label distribution is 996,077 Benign, 41,508 DoS-GoldenEye, and 10,990 DoS-Slowloris observations. The present study retains only Benign and DoS-GoldenEye traffic so that the large-scale reconstruction experiment focuses on a consistent, well-defined attack family.

Exact duplicate rows are removed while the Timestamp field is still present. This ordering is important because removing the timestamp first can cause distinct flows occurring at different times to become identical in the remaining feature space and therefore be removed as duplicates. After retaining the two required classes and removing exact duplicates, the dataset contains 994,414 Benign and 41,455 GoldenEye records.

Table 3.2: CSE-CIC-IDS2018 data preparation summary

Stage	**Rows**	**Description**
Raw file	1,048,575	Benign, GoldenEye, and Slowloris traffic.
Retain Benign + GoldenEye	1,037,585	10,990 Slowloris observations removed.
Remove exact duplicates	1,035,869	1,716 duplicates removed before dropping Timestamp.
Final Benign observations	994,414	Used to construct training, calibration, and evaluation pools.
Final GoldenEye observations	41,455	Used for development, contamination, and final holdout attack samples.

3.5 Handling Missing and Infinite Values

Network-flow features such as Flow Bytes/s and Flow Packets/s can contain infinite values when a rate is calculated for a flow with a zero or extremely small duration. These infinite values are converted to missing values before imputation. Importantly, the imputation statistics are derived from the training data and then applied to calibration, development, and final evaluation sets.

Median imputation is used because network-flow features often exhibit strong skewness and extreme values. Using training-derived medians reduces sensitivity to outliers while also preventing the final evaluation distribution from influencing preprocessing.

3.6 Feature Scaling

Different models receive transformations appropriate to their assumptions. Standard scaling is used for methods that depend directly on distances or margins, including Multivariate Gaussian and One-Class SVM. MinMax scaling is used for Autoencoder experiments so that neural network inputs occupy a bounded and comparable numerical range. Isolation Forest is evaluated using imputed numerical predictors without requiring distance-oriented scaling.

For UNSW-NB15, scaling is included in the pipelines of models that benefit from standardized numerical predictors, while tree-based models operate naturally on unscaled numerical values after categorical encoding.

3.7 Feature Selection

Mutual Information is used in the supervised experiments to estimate the dependence between predictors and the target labels [13]. On UNSW-NB15, the encoded predictors are ranked using Mutual Information and the top 60 features are retained in the selected-feature experiments.

For the unsupervised CSE-CIC study, feature selection does not rely on attack labels. Instead, pairwise correlations are estimated using the benign training data. Features with absolute correlation greater than 0.95 are removed in the selected-feature configuration. This reduces the usable feature space from 68 to 43 predictors.

3.8 Hyperparameter Optimization

Optuna is used to search model hyperparameters automatically [14]. The supervised UNSW experiments optimize the strongest tree-based ensemble models after initial baseline evaluation. The Autoencoder search includes encoding dimension, hidden-layer size, number of hidden layers, learning rate, dropout, L2 regularization, batch size, and the target false-positive rate used during threshold calibration.

Hyperparameter optimisation is performed using development data rather than the final test data. In the Autoencoder experiments, the objective is based on F1 performance across several anomaly prevalence levels so that the selected model is not optimized only for one particular test distribution.

3.9 Threshold Calibration

The Autoencoder outputs a reconstruction error rather than a direct class prediction. A threshold is therefore required to convert the continuous error into a binary benign/anomaly decision. The threshold is calibrated using a separate trusted benign calibration set rather than the final test labels.

This separation is important because choosing a threshold directly from the test labels would effectively optimize the model on the same data later reported as unseen evaluation data. By using independent benign calibration observations, the evaluation better resembles deployment, where labels for future traffic would not be available when the alert threshold is set.

3.10 Evaluation Metrics

Accuracy alone is insufficient for highly imbalanced anomaly-detection problems. Precision, Recall, F1-score, False Positive Rate, and False Negative Rate are therefore reported alongside accuracy [15].

Precision measures the fraction of predicted anomalies that are truly malicious, while Recall measures the fraction of malicious observations that are successfully detected. F1-score provides a harmonic balance between Precision and Recall. FPR measures the proportion of benign traffic incorrectly flagged as anomalous, whereas FNR measures the

proportion of attacks that remain undetected.

Chapter 4

Small Dataset Experiments

4.1 Experimental Setup

The first experimental stage uses the 1,000-row network-traffic dataset to study the effect of class imbalance and compare supervised and unsupervised methods. The dataset is divided into stratified training and testing partitions before any learned preprocessing. Supervised models include Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and XGBoost. SMOTE, class weighting, scaling, and feature selection are evaluated within the training workflow.

The unsupervised comparison includes Isolation Forest, Gaussian Mixture Model, Hidden Markov Model, Elliptic Envelope, and a Deep Autoencoder. These approaches attempt to detect anomalous observations without learning a conventional supervised attack boundary. The complete implementation of the small-dataset experiments is available in the project repository [16].

4.2 Supervised Results

Cross-validation experiments show that the small dataset contains only limited separation between the normal and anomaly classes. After model selection, the tuned Support Vector Machine provides the strongest final supervised F1-score on the untouched test partition. It achieves Accuracy of 0.5850, Precision of 0.1379, Recall of 0.6000, and F1-score of 0.2243.

Table 4.1: Final supervised performance on the small dataset

Model	**Accuracy**	**Precision**	**Recall**	**F1**
Support Vector Machine	0.5850	0.1379	0.6000	**0.2243**

The relatively high Recall compared with Precision indicates that the classifier detects a substantial fraction of anomalies but also produces many false positives. Because only 20 anomaly observations are present in the 200-row test partition, individual classification errors have a large effect on Precision and F1-score.

4.3 Unsupervised Results

The Autoencoder provides the strongest unsupervised F1-score at 0.1404, followed by Isolation Forest, Gaussian Mixture Model, Hidden Markov Model, and Elliptic Envelope. None of the unsupervised methods outperforms the tuned supervised SVM.

Table 4.2: Unsupervised performance on the small dataset

Model	**Recall**	**F1**
Autoencoder	0.2000	**0.1404**
Isolation Forest	0.1500	0.1154
Gaussian Mixture Model	0.1500	0.1111
Hidden Markov Model	0.1000	0.1081
Elliptic Envelope	0.1500	0.1071

4.4 Discussion

The results demonstrate that the small dataset is a difficult anomaly detection problem once preprocessing leakage is removed. The supervised model benefits directly from labelled anomaly examples, whereas the one-class methods depend entirely on the extent to which malicious traffic lies outside the learned normal distribution.

The modest scores are nevertheless useful because they provide a realistic baseline and demonstrate why experimental design is as important as algorithm selection. Applying SMOTE or scaling before the train-test split can produce significantly inflated results. By restricting these operations to the training pipeline, the reported scores provide a more defensible estimate of out-of-sample behaviour.

Chapter 5

Medium Dataset Experiments

5.1 Part A – Supervised Classification on UNSW-NB15

5.1.1 Experimental Design

The UNSW-NB15 stage evaluates seven classifiers: Logistic Regression, K-Nearest Neighbours, Linear SVM, Random Forest, XGBoost, LightGBM, and a soft Voting Classifier. The official training and test partitions are preserved. Model comparison, feature selection, and hyperparameter optimisation are performed only using the official training partition.

Initial baseline evaluation shows that tree-based ensemble models provide the strongest validation performance. Random Forest achieves a baseline F1-score of approximately 0.9706, LightGBM approximately 0.9700, and XGBoost approximately 0.9677. Mutual Information feature-selection experiments are then performed, and 60 encoded features are retained as a practical balance between complexity and predictive performance.

The complete UNSW-NB15 supervised classification workflow is available in the corresponding project notebook [17].

5.1.2 Hyperparameter Optimization

Optuna is used for 20-trial optimization of the strongest ensemble models. The best validation F1-score is obtained by LightGBM at 0.9733, followed by XGBoost at 0.9721 and Random Forest at 0.9714. A soft Voting Classifier combining the strongest models achieves an F1-score of 0.9731.

Table 5.1: Best internal validation results on UNSW-NB15

Model	**Validation F1**
LightGBM	**0.9733**
Soft Voting Classifier	0.9731
XGBoost	0.9721
Random Forest	0.9714

5.1.3 Final Official Test Performance

LightGBM is selected based on the development results. The preprocessing pipeline, feature-selection procedure, and classifier are then refitted using the complete official training set. The official test partition, which has not been used during feature selection or hyperparameter optimization, is evaluated once.

Table 5.2: Final LightGBM performance on the official UNSW-NB15 test set

Metric	**Value**
Accuracy	0.8696
Precision	0.8196
Recall	**0.9787**
F1-score	**0.8921**

The official test result is lower than the internal validation result, which highlights the importance of maintaining a genuinely independent test partition. The high Recall of 0.9787 indicates that LightGBM detects nearly all attacks in the official test set, while Precision of 0.8196 indicates a higher number of false-positive attack predictions. The resulting F1-score of 0.8921 nevertheless represents strong supervised performance on the benchmark.

5.2 Part B – Unsupervised Anomaly Detection on CSE-CIC-IDS2018

5.2.1 Experimental Setup

The medium-scale unsupervised experiment trains models using 100,000 benign CSE-CIC observations. A separate set of 20,000 benign observations is reserved for threshold calibration. Following training-only imputation and constant-feature removal, 68 usable predictors remain.

Three development sets are constructed at anomaly prevalence levels of 0.2%, 1%, and 5%. These development sets are used for model selection and hyperparameter optimisation. Separate final holdout sets of 30,000 observations at the same prevalence levels are reserved and remain untouched until the selected model has been finalized.

5.2.2 Models Evaluated

Four unsupervised anomaly-detection methods are evaluated: Multivariate Gaussian, Isolation Forest, One-Class SVM, and Deep Autoencoder. Each method represents a different modelling principle. The Multivariate Gaussian approach measures deviation from a fitted normal distribution, Isolation Forest identifies points that are easy to isolate, One-Class SVM learns a boundary surrounding normal observations, and the Autoencoder uses reconstruction error as its anomaly score.

A selected-feature experiment is also performed using a benign-only correlation filter. Features with absolute pairwise correlation above 0.95 are removed, reducing the predictor count from 68 to 43. The implementation of the medium-scale unsupervised experiments is available in the project repository [18].

5.2.3 Autoencoder Model Selection

The Autoencoder consistently provides the strongest overall development performance. The feature-selected Autoencoder improves over its all-feature baseline, but Optuna optimization of the complete feature space produces the best final development model.

The selected all-feature Autoencoder uses an encoding dimension of 8, hidden dimension

of 128, two hidden layers, learning rate of approximately 5.85×10^{-4}, dropout of approximately 0.0148, small L2 regularization, batch size 1024, and a threshold-calibration target FPR near 0.0193.

5.2.4 Final Holdout Results

The selected Autoencoder is evaluated on fresh 30,000-row holdout sets. It achieves Recall of 1.0000 at all three anomaly prevalence levels. Precision increases substantially as anomaly prevalence rises because the approximately 2.1% benign false-positive rate represents a smaller proportion of the predicted anomaly population.

Table 5.3: Final Autoencoder performance on medium-scale CSE-CIC-IDS2018 holdouts

Anomaly Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9786	0.0856	1.0000	0.1577	0.0214	0.0000
1%	0.9789	0.3212	1.0000	0.4862	0.0213	0.0000
5%	0.9799	0.7136	1.0000	**0.8329**	0.0211	0.0000

At 5% anomaly prevalence, all 1,500 GoldenEye observations in the final holdout are detected. The result supports the selection of the Autoencoder for the large-scale stage while also demonstrating why Precision must be interpreted together with prevalence. At 0.2% prevalence, even a modest FPR causes false positives to outnumber the small number of true attacks.

5.3 Comparison of the Medium-Scale Studies

The two medium-scale experiments address fundamentally different learning conditions. UNSW-NB15 represents a supervised setting in which attack labels are available during training, allowing LightGBM to learn a direct normal-versus-attack classification boundary. CSE-CIC-IDS2018 is deliberately treated as a one-class anomaly detection problem in which attacks are excluded from model training.

The supervised LightGBM model provides strong benchmark classification, while the Autoencoder demonstrates that useful detection is possible without attack labels during fitting. Since the objective of the final stage is to investigate one-class anomaly detection

at scale, the Autoencoder is selected for detailed large-scale analysis.

Chapter 6

Large Dataset Experiments

6.1 Introduction

The large-scale experiments constitute the primary investigation of the internship. Rather than comparing many different algorithms, this stage focuses on the behaviour of the Deep Autoencoder when the amount and composition of its training data change.

Two major questions are studied. The first is whether increasing the amount of representative benign training data improves anomaly detection on GoldenEye traffic. The second is how strongly the model degrades when attack observations become part of the nominally normal training distribution.

6.2 Large-Scale Data Preparation

After retaining Benign and GoldenEye traffic and removing exact duplicates while the Timestamp field remains available, the dataset contains 1,035,869 observations. Timestamp is then removed because it may encode capture schedule information rather than intrinsic network behaviour.

The final class distribution contains 994,414 benign observations and 41,455 GoldenEye observations. A clean benign training pool, clean calibration data, separate development pools, and separate final holdout pools are constructed before model development.

6.3 Benign-Only Training Experiment

6.3.1 Training and Calibration

The benign-only Autoencoder uses 300,000 benign flows for model training. A further 30,000 benign flows are reserved exclusively for threshold calibration. Remaining benign and attack observations are divided into development and final evaluation pools so that the final holdouts are not involved in architecture or threshold selection.

The complete large-scale benign-only Autoencoder experiment is available in the corresponding notebook [19].

The final evaluation contains three 30,000-row holdouts:

Table 6.1: Composition of final large-scale holdout sets

Anomaly Rate	Benign	GoldenEye	Total
0.2%	29,940	60	30,000
1%	29,700	300	30,000
5%	28,500	1,500	30,000

6.3.2 Hyperparameter Optimization

A baseline Autoencoder is first evaluated to establish reference performance. Optuna is subsequently used for 20 hyperparameter trials. The best configuration uses an encoding dimension of 8, hidden dimension of 192, two hidden layers, learning rate approximately 5.97×10^{-4}, dropout approximately 0.0818, L2 regularization approximately 1.34×10^{-7}, batch size 2048, and a target calibration FPR of approximately 0.0124.

The tuned model achieves development F1-scores of 0.2516, 0.6289, and 0.8991 on the 0.2%, 1%, and 5% development sets, giving a mean development F1 of approximately 0.5932.

6.3.3 Final Benign-Only Results

The optimized model is evaluated on the untouched final holdouts after all development decisions are complete.

Table 6.2: Final benign-only Autoencoder performance

Anomaly Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9883	0.1460	1.0000	0.2548	0.0117	0.0000
1%	0.9884	0.4622	1.0000	0.6322	0.0118	0.0000
5%	0.9887	**0.8171**	0.9980	**0.8986**	0.0118	0.0020

The final mean F1-score across the three prevalence levels is 0.5952. At 5% prevalence, the model detects 99.8% of GoldenEye attacks while maintaining a false-positive rate of approximately 1.18%. The close agreement between development F1 and final F1 suggests that the selected model generalises consistently to the independent holdout pool.

6.4 Training Contamination Experiment

6.4.1 Motivation

One-class anomaly detection commonly assumes that the training dataset represents normal behaviour. In operational environments, however, large collections of historical traffic may contain undetected attacks. Because an Autoencoder is trained to reconstruct the samples it sees, sufficiently frequent attacks may gradually become part of the learned reconstruction distribution.

The contamination experiment investigates this behaviour directly by training three Autoencoders using progressively larger proportions of GoldenEye traffic. The total training size is held constant at 300,000 observations so that increasing contamination replaces part of the benign training population instead of simply increasing the total training size. The implementation of these experiments is available in the project repository [20].

6.4.2 Training Dataset Composition

Table 6.3: Composition of contaminated Autoencoder training sets

Contamination	Benign	GoldenEye	Total
0.2%	299,400	600	300,000
1%	297,000	3,000	300,000
5%	285,000	15,000	300,000

The anomaly threshold remains calibrated using trusted clean benign traffic. Separate attack and benign pools are retained for development and final evaluation.

A methodological limitation should be noted. The contaminated conditions use independently sampled benign cohorts rather than replacing attacks within the exact same 300,000-row benign reference set used in the clean experiment. The experiment therefore provides a robustness comparison, but the performance difference cannot be attributed exclusively to contamination with the strictness of a fully paired causal design. A paired replacement experiment is recommended as future work.

6.4.3 Final Contaminated-Training Results

Table 6.4: Final holdout performance under contaminated training

Train Contam.	Test Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.2%	0.9694	0.0337	0.5167	0.0633	0.0297	0.4833
0.2%	1%	0.9657	0.1478	0.5100	0.2292	0.0297	0.4900
0.2%	5%	0.9489	0.4900	0.5380	0.5129	0.0295	0.4620
1%	0.2%	0.9800	0.0073	0.0667	0.0131	0.0182	0.9333
1%	1%	0.9728	0.0409	0.0767	0.0533	0.0182	0.9233
1%	5%	0.9367	0.1891	0.0807	0.1131	0.0182	0.9193
5%	0.2%	0.9703	0.0000	0.0000	0.0000	0.0278	1.0000
5%	1%	0.9626	0.0024	0.0067	0.0035	0.0278	0.9933
5%	5%	0.9244	0.0235	0.0127	0.0165	0.0276	0.9873

6.5 Comparison Between Clean and Contaminated Training

The effect of contamination becomes particularly clear when mean F1 across all final holdouts and performance on the 5% final holdout are compared directly.

Table 6.5: Comparison of clean and contaminated Autoencoder training

Training Condition	Mean Final F1	5% Precision	5% Recall	5% F1
Clean Benign	**0.5952**	**0.8171**	**0.9980**	**0.8986**
0.2% Contamination	0.2684	0.4900	0.5380	0.5129
1% Contamination	0.0598	0.1891	0.0807	0.1131
5% Contamination	0.0067	0.0235	0.0127	0.0165

Even 0.2% contamination produces a substantial reduction in detection performance. The mean final F1 falls from 0.5952 to 0.2684, and the 5% holdout Recall falls from 0.9980 to 0.5380. At 1% contamination, the model misses more than 91% of attacks on the 5% final holdout. At 5% contamination, attack Recall is only 0.0127 and the FNR reaches 0.9873.

These results are consistent with the reconstruction objective of an Autoencoder. The network is optimized to reduce reconstruction error for observations included in its training distribution. As GoldenEye traffic becomes more frequent in training, the model increasingly learns representations that reconstruct GoldenEye observations rather than treating them as unfamiliar behaviour. The separation between benign and attack reconstruction errors therefore collapses.

6.6 Discussion

The large-scale study demonstrates that increasing representative benign training data can produce strong unsupervised anomaly-detection performance, but it also shows that training-data quality is critical. The benign-only Autoencoder achieves excellent Recall at the 5% prevalence level while maintaining a relatively low false-positive rate.

The contamination results provide an important operational lesson. A reconstruction

model should not be assumed to remain robust simply because attacks form a numerically small proportion of a large dataset. Even low levels of repeated attack behaviour can affect the learned normal representation. Practical deployment therefore requires careful construction and monitoring of the benign reference dataset, periodic validation, and mechanisms for detecting contamination or distribution shift.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This Summer Research Internship investigated supervised, unsupervised, and deep-learning approaches to network anomaly detection across datasets of increasing scale and complexity. The work began with a small 1,000-row dataset and progressed through UNSW-NB15 and CSE-CIC-IDS2018 to a large-scale Deep Autoencoder study. Throughout the project, particular emphasis was placed on preventing information leakage by separating training, preprocessing, threshold calibration, model development, and final evaluation data.

On the small dataset, the tuned Support Vector Machine achieved an F1-score of 0.2243 with Recall of 0.6000. The strongest unsupervised model, the Autoencoder, achieved F1 = 0.1404. These scores provide a realistic representation of performance on genuinely unseen data after restricting learned preprocessing and resampling to the training workflow.

The supervised UNSW-NB15 study demonstrated the strength of modern tree-based ensemble methods when representative labels are available. After feature-selection experiments and Optuna optimization, LightGBM was selected and achieved Accuracy of 0.8696, Precision of 0.8196, Recall of 0.9787, and F1-score of 0.8921 on the untouched official test partition.

The medium-scale CSE-CIC-IDS2018 experiment showed that a Deep Autoencoder can provide strong one-class attack detection without using attack labels during model fitting. The selected Autoencoder achieved a final 5% holdout F1-score of 0.8329 and detected all 1,500 GoldenEye attacks in that evaluation set.

The large-scale benign-only Autoencoder produced the strongest unsupervised result of the internship. Trained using 300,000 benign flows and calibrated on a separate 30,000-row benign set, it achieved an F1-score of 0.8986 with Precision of 0.8171, Recall of 0.9980, FPR of 0.0118, and FNR of 0.0020 on the final 5% anomaly holdout.

The training-contamination experiment revealed the principal limitation of reconstruction-based anomaly detection observed in this study. Mean final F1 decreased from 0.5952 under clean benign training to 0.2684 with 0.2% contamination, 0.0598 with 1% contamination, and 0.0067 with 5% contamination. The results therefore demonstrate that Autoencoders can be highly effective when trained on representative normal traffic, but their ability to distinguish malicious behaviour can deteriorate rapidly when attack patterns are repeatedly presented during training.

Overall, the internship demonstrates that model performance cannot be separated from experimental methodology. Reliable anomaly detection requires not only an appropriate algorithm but also leakage-safe preprocessing, representative training data, independent calibration, careful threshold selection, class-imbalance-aware evaluation, and genuinely untouched final test data.

7.2 Limitations

The large-scale experiments focus on a single CSE-CIC-IDS2018 capture file and one attack family, DoS-GoldenEye. The resulting performance therefore should not be interpreted as evidence that the same Autoencoder will detect every intrusion category equally well. Different attacks may occupy very different regions of feature space and may produce reconstruction-error distributions that overlap more strongly with benign traffic.

Autoencoder training and Optuna optimization are also stochastic. Although fixed random states are used where possible, repeating the full large-scale study across several independent random seeds would provide stronger estimates of variance and confidence intervals.

A further limitation concerns the contamination experiment. The clean and contaminated models use independently sampled benign training cohorts. The observed degradation with increasing contamination is large and systematic, but a fully paired experiment would provide a stronger causal comparison by beginning with the same fixed benign training sample and replacing an exact fraction of those observations with attacks.

7.3 Future Work

Future work can extend the current framework to multiple attack families and additional benchmark datasets such as CICIDS2017 and TON_IoT. Evaluating multiple CSE-CIC-IDS2018 capture days would also help determine how well a model trained on one traffic distribution generalises to another day or network condition.

More advanced representation-learning approaches can also be explored, including Variational Autoencoders, Transformer-based Autoencoders, contrastive learning, Graph Neural Networks, and hybrid deep-learning architectures. Such methods may learn richer representations of network behaviour and improve detection of subtle attacks.

Adaptive thresholding represents another important direction. The optimal reconstruction-error threshold may change when legitimate network traffic evolves, so drift-detection and online calibration methods could be used to update thresholds while protecting the normal reference distribution from malicious contamination.

A stronger contamination experiment should use the same fixed 300,000-row benign training set for every condition and replace an exact number of those observations with attack flows. Repeating this paired experiment across multiple seeds would make it possible to quantify the contamination effect with confidence intervals.

Finally, explainable anomaly-detection methods can be incorporated to identify which features contribute most strongly to a high anomaly score. Such explanations would make model alerts more useful to human security analysts and improve the practical deployability of machine-learning-based intrusion detection systems.

Bibliography

[1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[2] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[3] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Communications of the ACM*, vol. 55, no. 2, pp. 55–61, 2012.

[4] N. Moustafa and J. Slay, "Unsw-nb15: A comprehensive data set for network intrusion detection systems," in *2015 Military Communications and Information Systems Conference*, 2015, pp. 1–6.

[5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116.

[6] R. Kumar, "Anomaly detection in computer log using machine learning techniques," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques, 2026, gitHub repository containing project notebooks, results, datasets, and documentation.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422.

[11] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, Massachusetts: MIT Press, 2016.

[13] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Physical Review E*, vol. 69, no. 6, 2004.

[14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[15] D. M. W. Powers, "Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

[16] R. Kumar, "Small dataset anomaly detection experiments," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques/blob/main/notebooks/Small_Dataset.ipynb, 2026, jupyter Notebook.

[17] ——, "Unsw-nb15 supervised classification experiments," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques/blob/main/notebooks/Medium_Dataset_1.ipynb, 2026, jupyter Notebook.

[18] ——, "Cse-cic-ids2018 unsupervised anomaly detection experiments," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques/blob/main/notebooks/Medium_Dataset_2.ipynb, 2026, jupyter Notebook.

[19] ——, "Large-scale benign-only autoencoder experiments," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques/blob/main/notebooks/Large_Dataset_Benign.ipynb, 2026, jupyter Notebook.

[20] ——, "Large-scale autoencoder training contamination experiments," https://github.com/RajolKumar2003/Anomaly-Detection-in-Computer-Log-Using-ML-Techniques/blob/main/notebooks/Large_Dataset_Contamination.ipynb, 2026, jupyter Notebook.